

EXISTING SYSTEM

Existing personal productivity systems generally depend on separate applications for managing tasks, calendars, notes, reminders, emails, and other daily activities. Users must manually open each application and perform the required operation. This fragmented approach makes it difficult to manage multiple activities from a single interface.

Traditional productivity applications are usually designed around predefined features and workflows. They may provide task management or calendar functionality, but they often have limited ability to understand natural-language instructions and execute actions across different services. When users need to connect multiple applications, separate APIs and custom integration logic are generally required.

Existing AI assistants provide conversational interaction and can understand user queries, but their ability to perform actions is often restricted by the tools and integrations available to them. Adding new services can require additional development and configuration. As a result, existing systems may not provide a flexible, standardized, and easily extensible architecture for connecting an AI agent with multiple productivity tools.

PROPOSED SYSTEM

The proposed system, “MCP-Based Personal Productivity Agent,” is designed to provide a unified and intelligent interface for managing personal productivity activities. The system uses an AI-based agent that understands natural-language instructions and determines the appropriate action to perform.

The Model Context Protocol (MCP) is used as the communication layer between the AI agent and external tools or services. Through MCP, productivity tools can expose their capabilities in a standardized manner, allowing the agent to discover and invoke the required tools without requiring a completely different integration mechanism for each service.

For example, a user can provide a request such as “Create a task to complete my project report tomorrow.” The AI agent interprets the request, identifies the appropriate productivity tool, and sends the required operation through the MCP interface. Similarly, the system can be extended to support calendar events, reminders, notes, information retrieval, and other productivity-related operations.

The proposed system aims to reduce repetitive manual work, minimize application switching, improve productivity, and provide a natural conversational experience. Its modular architecture also makes it easier to add new tools and services in the future.

HARDWARE REQUIREMENTS

The proposed MCP-Based Personal Productivity Agent does not require specialized hardware and can be developed and executed on a standard computer system. The recommended hardware requirements are:

- Processor: Intel Core i5 or equivalent AMD processor and above
- RAM: Minimum 8 GB; 16 GB recommended for development and testing
- Storage: Minimum 256 GB SSD
- Display: 1366 × 768 resolution or higher
- Network: Stable broadband or Wi-Fi connection
- Input Devices: Standard keyboard and mouse
- Optional: Microphone for future voice-based interaction

The system may require a stable internet connection when using cloud-based Large Language Models (LLMs), external APIs, or remote MCP services.

SOFTWARE REQUIREMENTS

The software requirements for developing and operating the MCP-Based Personal Productivity Agent include:

- Operating System: Windows 10/11, Linux, or macOS
- Programming Language: Python
- Development Environment: Visual Studio Code or another suitable IDE
- AI/LLM Integration: An API or locally hosted Large Language Model
- MCP: Model Context Protocol-compatible client and MCP servers
- Backend Framework: FastAPI, Flask, or an equivalent Python framework
- Database: SQLite, PostgreSQL, or another suitable database depending on implementation
- Version Control: Git and GitHub
- API Testing: Postman or equivalent tool
- Web Browser: Google Chrome, Microsoft Edge, Mozilla Firefox, or equivalent
- Internet Connection: Required for cloud-based AI models and external productivity services

The exact software stack can be selected according to the implementation requirements and the productivity tools integrated into the system.